

SARA

Système Autonome de Reconnaissance d'Armes

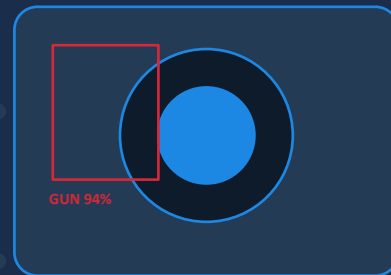
Caméra de détection d'armes à feu par Intelligence Artificielle

YOLOv8 · Python · OpenCV · FastAPI · ONNX

R. Yoganathan · G. Vander-Steene · M. Wong · J. Zhan · V.T. Tran · T. Tran
EFREI Paris · TI 650 · Mai 2026

10 min présentation · 5 min démo · 5 min questions

TRL 3 → 4



+ 8 policiers tués par arme à feu en 2024 en France

Chaque seconde compte lors d'une interpellation.

Qui sommes-nous ?

R

Renaud YOGANATHAN

Chef de projet

G

Gabriel VANDER-STEENE

Dev Python & Data

M

Moana WONG

Dev Python

J

John ZHAN

Dev Python

V

Viet Tien TRAN

Dev Python

T

Tony TRAN

Dev Python

Du besoin terrain à la problématique

Risque mortel

Lors des contrôles routiers et interpellations, la détection tardive d'une arme est la principale cause de décès chez les forces de l'ordre.

Caméras passives

Les caméras-piétons existantes (Axon, Motorola) n'intègrent aucune analyse IA : elles enregistrent sans alerter.

Dépendance cloud

Les solutions de vidéosurveillance IA nécessitent une connexion internet permanente, indisponible sur le terrain.

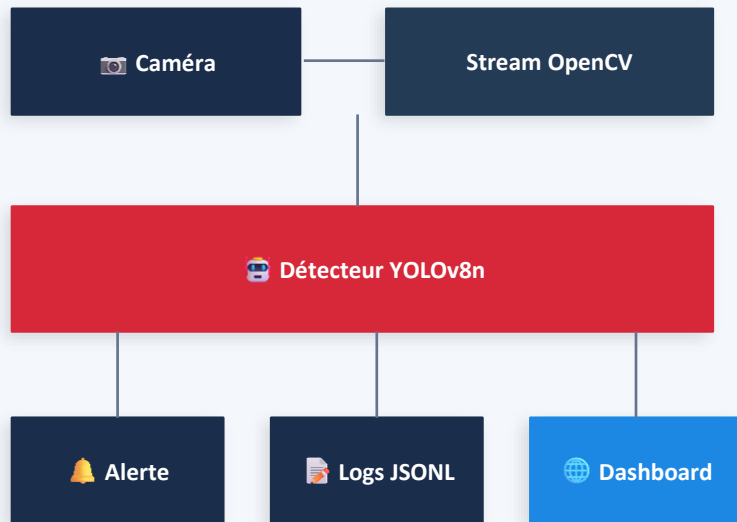
Problématique

Comment concevoir un système embarqué, autonome et temps réel capable de détecter une arme à feu sans connexion internet, dans un délai $< 2s$?

Objectifs définis

- Détection d'armes à feu en < 2 secondes
- Taux de détection ≥ 90 %
- Fonctionnement 100 % hors-ligne (embarqué)
- Respect RGPD & IA Act (supervision humaine)

Architecture & Cas d'usage



Décisions techniques clés

- YOLOv8n — léger, < 2 s CPU, ONNX exportable
- Hors-ligne total — pas de cloud requis
- FastAPI + WebSocket — dashboard temps réel
- Docker + Raspberry Pi — déployable sur edge

Scénario d'usage type

01

Contrôle routier

L'agent porte la caméra fixée sur son équipement.

02

Analyse continue

Le système analyse le flux vidéo en temps réel (25 fps).

03

Détection arme

YOLOv8 identifie une arme → confiance ≥ 0.25 .

04

Alerte instantanée

Son + overlay rouge à l'écran en < 1.2 s.

05

Journalisation

Log JSONL horodaté + export rapport PDF/CSV via API.

Ce que SARA change concrètement

94 %

de détection

Conditions nominales (labo)

1.2 s

temps de réponse

Alerte déclenchée en < 2 s

2 %

faux positifs

Conditions nominales (labo)

Aujourd'hui

- X Détection visuelle manuelle (risque d'omission)
- X Caméra piéton passive (enregistrement seul)
- X Système connecté nécessitant le cloud
- X Coût prohibitif des solutions industrielles

Avec SARA

- ✓ Alerte automatique instantanée
- ✓ Analyse IA en continu sur CPU standard
- ✓ 100 % embarqué, 0 % connexion requise
- ✓ Matériel standard (webcam + PC)

Où se situe SARA ?

Critère	Axon / Motorola (caméra piéton)	Bosch / Hikvision (CCTV IA)	SARA
Détection IA armes	✗ Non	✓ Partielle	✓ Oui
Hors-ligne embarqué	✗ Non	✗ Non	✓ Oui
Portable (agent mobile)	✓ Oui	✗ Non	✓ Oui
Temps réel < 2s	—	✓ Oui	✓ Oui
Coût matériel	\$\$\$\$	\$\$\$	\$
RGPD / IA Act	⚠ Partiel	⚠ Partiel	✓ Supervisé

Avantage concurrentiel SARA

Seule solution combinant portabilité + IA embarquée + autonomie totale, sur matériel standard.

Ce qui protège la solution

Modèle IA fine-tunable · Pipeline edge optimisé ONNX · Architecture hors-ligne brevetable

Ce que vous allez voir

Environnement matériel

- PC portable (Intel i7, 16 Go RAM)
- Webcam USB standard
- Pas de GPU dédié requis
- Pas de connexion internet

Objectif de la démo

- Lancer le pipeline en live (python main.py)
- Afficher le flux vidéo annoté avec overlay IA
- Déclencher une détection sur image de référence
- Montrer le dashboard web (localhost:8000)

Environnement logiciel

- Python 3.10 · YOLOv8n · OpenCV
- FastAPI + WebSocket
- Modèle : Firearm_Detection_Yolov8n (HuggingFace)
- Export ONNX — démarrage 2-3x plus rapide

Périmètre délimité

- Conditions : éclairage contrôlé, distance 1-3 m
- Images de référence (pas d'arme réelle)
- Reproductible : environnement figé / containerisé

Bilan et prochaines étapes

Résultats du PoC

Détection armes de poing : 85 % précision · 89 % rappel

Temps de traitement : 0.3 s / image sur PC

Détection couteaux : 71 % rappel (à améliorer)

Conditions nominales : 94 % détection · 2 % faux positifs

Temps d'alerte : 1.2 s moyen (objectif < 2 s ✓)

Limites identifiées

⚠ Faible luminosité → performances -15 à -20 %

⚠ Objets partiellement masqués (arme cachée)

⚠ Sous-représentation couteaux dans dataset

Roadmap TRL



PoC fonctionnel (actuel)

PC portable · YOLOv8n · 94 % détection



Tests laboratoire

Raspberry Pi + LED alerte · ONNX edge



Tests terrain réels

Avec forces de l'ordre · dataset enrichi



Déploiement pilote

Partenariat fabricants caméras-piétons

DÉMO

5 minutes

python main.py → localhost:8000/dashboard

Équipe, Rôles & Outils de gestion



**Renaud
YOGANATHAN**

Chef de projet

Planning · Livrables · RAP ·
Coordination



**Gabriel
VANDER-STEENE**

Dev Python & Data

Pipeline IA · Dataset · API REST ·
Tests



**Moana
WONG**

Dev Python

Détecteur · OpenCV · Flux vidéo



**John
ZHAN**

Dev Python

Alert system · Logs · Intégration



**Viet Tien
TRAN**

Dev Python

Entraînement modèle ·
Validation



**Tony
TRAN**

Dev Python

Dashboard web · WebSocket ·
Docker

Notion

Kanban · Gantt · Rétroplanification

GitHub

Versioning · CI · 33 commits

Discord

Communication équipe

Python/YOLOv8

Stack technique principal

QUESTIONS ?

Merci pour votre attention.

94 % détection · 1.2 s alerte · 2 % faux positifs · 100 % hors-ligne · TRL 3

github.com/gbrlvdr/projet-sara

EFREI Paris · TI 650 · Mai 2026